Example:
Test Case 2: Purchase Request Submission Test

Code

```
import os
import pathlib
import pytest
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.support.ui import Select

# Paths and configuration
BASE_DIR = pathlib.Path(__file__).resolve().parent.parent
SCREENSHOT_DIR = BASE_DIR / "test-results" / "screenshots"
SCREENSHOT_DIR.mkdir(parents=True, exist_ok=True)

# Test Configuration - configurable via environment variables
BASE_URL = os.environ.get("BASE_URL", "http://localhost:3000")
LOGIN_URL = f"{BASE_URL}/login"
CREATE_REQ_URL = f"{BASE_URL}/staff/create-request"
REQUESTS_LIST_URL = f"{BASE_URL}/staff/requests"
STAFF_USERNAME = os.environ.get("TEST_STAFF_USERNAME", "test_staff")
STAFF_PASSWORD = os.environ.get("TEST_STAFF_PASSWORD", "TestStaff@2026")
HEADLESS_MODE = os.environ.get("HEADLESS", "true").lower() in ("true", "1", "yes")


class TestPurchaseRequest:
    """
    Test Case 2: ProcuraMed Purchase Request Submission Test Suite
    Covers end-to-end requisition creation, item specification, submission verification,
    and form field validation error handling.
    """

    def setup_method(self, method):
        """Setup Chrome WebDriver before each test execution."""
        options = Options()
        if HEADLESS_MODE:
            options.add_argument("--headless=new")
        options.add_argument("--window-size=1920,1080")
        options.add_argument("--no-sandbox")
        options.add_argument("--disable-dev-shm-usage")
        options.add_argument("--disable-gpu")
        options.add_argument("--ignore-certificate-errors")

        self.driver = webdriver.Chrome(options=options)
        self.driver.maximize_window()
        self.wait = WebDriverWait(self.driver, 12)

    def teardown_method(self, method):
        """Tear down and close the WebDriver session after each test."""
        if hasattr(self, "driver") and self.driver:
            self.driver.quit()

    def click_element(self, element):
        """Helper to reliably scroll and click element across animations."""
        self.driver.execute_script("arguments[0].scrollIntoView({block: 'center'});", element)
        try:
            element.click()
        except Exception:
            self.driver.execute_script("arguments[0].click();", element)

    def login_as_staff(self):
        """Helper to perform authenticated login as Department Staff."""
        driver = self.driver
        wait = self.wait
```

```
        driver.get(LOGIN_URL)
        username_field = wait.until(EC.visibility_of_element_located((By.NAME, "username")))
        username_field.clear()
        username_field.send_keys(STAFF_USERNAME)

        password_field = wait.until(EC.visibility_of_element_located((By.NAME, "password")))
        password_field.clear()
        password_field.send_keys(STAFF_PASSWORD)

        login_btn = wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, "button[type='submit']")))
        self.click_element(login_btn)

        wait.until(EC.url_contains("/staff/dashboard"))

    def test_submit_purchase_request_success(self):
        """
        Test Case 2: Submit a valid Purchase Requisition.
        Steps:
        1. Login as authorized Department Staff.
        2. Navigate to Create Purchase Requisition page.
        3. Fill Step 1 Requisition Details (Title, Priority, Clinical Justification).
        4. Fill Step 2 Item Specifications (Product Name, Category, Quantity, Unit Price, Specs).
        5. Review Requisition Summary in Step 3 and submit.
        6. Verify confirmation alert and automatic redirection to My Requisitions list.
        """
        driver = self.driver
        wait = self.wait

        # Step 1: Login as staff requester
        self.login_as_staff()
        driver.save_screenshot(str(SCREENSHOT_DIR / "tc2_step1_staff_dashboard.png"))

        # Step 2: Open Create Purchase Requisition Form
        driver.get(CREATE_REQ_URL)
        wait.until(EC.url_contains("/staff/create-request"))
        form_heading = wait.until(
            EC.visibility_of_element_located((By.XPATH, "//h2[contains(text(), 'Step 1: General Requisition Details')]"))
        )
        assert form_heading.is_displayed(), "Create Requisition form failed to load"
        driver.save_screenshot(str(SCREENSHOT_DIR / "tc2_step2_create_form_loaded.png"))

        # Step 3: Enter Step 1 General Requisition Details
        title_val = "Automated Procurement - Emergency Surgical Gloves Replenishment"
        title_input = wait.until(
            EC.visibility_of_element_located((By.XPATH, "//input[contains(@placeholder, 'Monthly Replenishment')]"))
        )
        title_input.clear()
        title_input.send_keys(title_val)

        priority_select_elem = wait.until(
            EC.visibility_of_element_located((By.XPATH, "//label[contains(text(), 'Priority Level')]/following::select[1]"))
        )
        Select(priority_select_elem).select_by_value("HIGH")

        justification_val = "Emergency stock replenishment required for Operation Theatre surgical procedures."
        justification_input = wait.until(EC.visibility_of_element_located((By.TAG_NAME, "textarea")))
        justification_input.clear()
        justification_input.send_keys(justification_val)

        driver.save_screenshot(str(SCREENSHOT_DIR / "tc2_step3_step1_details_entered.png"))

        next_btn_1 = wait.until(EC.element_to_be_clickable((By.XPATH, "//button[contains(., 'Next: Add Items & Specs')]")))
        self.click_element(next_btn_1)

        # Step 4: Enter Step 2 Multi-Item Specifications
        wait.until(
            EC.visibility_of_element_located((By.XPATH, "//h2[contains(text(), 'Step 2: Add Requisition Items')]"))
        )

        item_name_input = wait.until(
```

```
            EC.visibility_of_element_located((By.XPATH, "//input[contains(@placeholder, 'Surgical Gloves')]"))
        )
        item_name_input.clear()
        item_name_input.send_keys("Nitrile Surgical Gloves Powder-Free (Box of 100)")

        category_elem = wait.until(
            EC.visibility_of_element_located((By.XPATH, "//label[contains(text(), 'Category')]/following::select[1]"))
        )
        wait.until(lambda d: len(Select(category_elem).options) > 1)
        Select(category_elem).select_by_visible_text("Medical Consumables")

        qty_input = wait.until(
            EC.visibility_of_element_located((By.XPATH, "//label[contains(text(), 'Quantity')]/following::input[1]"))
        )
        qty_input.clear()
        qty_input.send_keys("50")

        price_input = wait.until(
            EC.visibility_of_element_located((By.XPATH, "//label[contains(text(), 'Estimated Unit Price')]/following::input
        )
        price_input.clear()
        price_input.send_keys("450.00")

        specs_input = wait.until(
            EC.visibility_of_element_located((By.XPATH, "//input[contains(@placeholder, 'Nitrile, textured grip')]"))
        )
        specs_input.clear()
        specs_input.send_keys("Medical grade nitrile, powder-free, hypoallergenic, ISO certified.")

        driver.save_screenshot(str(SCREENSHOT_DIR / "tc2_step4_item_specs_entered.png"))

        next_btn_2 = wait.until(EC.element_to_be_clickable((By.XPATH, "//button[contains(., 'Next: Review Requisition')]"))
        self.click_element(next_btn_2)

        # Step 5: Review & Submit
        wait.until(
            EC.visibility_of_element_located((By.XPATH, "//h2[contains(text(), 'Step 3: Review Requisition Before Submissio
        )
        driver.save_screenshot(str(SCREENSHOT_DIR / "tc2_step5_review_summary.png"))

        submit_btn = wait.until(EC.element_to_be_clickable((By.XPATH, "//button[contains(., 'Submit Requisition')]")))
        self.click_element(submit_btn)

        # Step 6: Verify Confirmation Banner
        success_banner = wait.until(
            EC.visibility_of_element_located((By.XPATH, "//div[contains(@class, 'bg-emerald-50')]"))
        )
        assert "Purchase Requisition submitted successfully" in success_banner.text, (
            f"Expected success message, got: {success_banner.text}"
        )
        driver.save_screenshot(str(SCREENSHOT_DIR / "tc2_step6_submission_confirmation.png"))

        # Step 7: Verify Redirection to My Requisitions and entry in list
        wait.until(EC.url_contains("/staff/requests"))
        requisition_row = wait.until(
            EC.visibility_of_element_located((By.XPATH, "//table//tbody//tr[1]"))
        )
        driver.save_screenshot(str(SCREENSHOT_DIR / "tc2_step7_requisition_in_list.png"))

        assert "Submitted" in requisition_row.text, f"Requisition status not 'Submitted': {requisition_row.text}"

    def test_purchase_request_validation_errors(self):
        """
        Negative Test: Attempt proceeding with empty required fields.
        Verify that proper validation error messages appear for Title and Justification.
        """
        driver = self.driver
        wait = self.wait

        self.login_as_staff()
```

```
driver.get(CREATE_REQ_URL)
wait.until(EC.url_contains("/staff/create-request"))

# Click Next without filling mandatory fields
next_btn_1 = wait.until(EC.element_to_be_clickable((By.XPATH, "//button[contains(., 'Next: Add Items & Specs')]")))
self.click_element(next_btn_1)

title_error = wait.until(
    EC.visibility_of_element_located((By.XPATH, "//*[contains(text(), 'Requisition title / purpose is required.')]"
)
justification_error = wait.until(
    EC.visibility_of_element_located((By.XPATH, "//*[contains(text(), 'Reason and justification are required.')]"))
)

driver.save_screenshot(str(SCREENSHOT_DIR / "tc2_validation_errors_displayed.png"))

assert title_error.is_displayed(), "Title validation error not shown"
assert justification_error.is_displayed(), "Justification validation error not shown"
```

Screenshot

```
PS F:\ProcuraMed> pytest tests/test_purchase_request.py -v
============================= test session starts =============================
platform win32 -- Python 3.13.5, pytest-9.1.1, pluggy-1.7.0
rootdir: F:\ProcuraMed
collected 2 items

tests/test_purchase_request.py::TestPurchaseRequest::test_submit_purchase_request_success PASSED [ 50%]
tests/test_purchase_request.py::TestPurchaseRequest::test_purchase_request_validation_errors PASSED [100%]

============================== 2 passed in 27.78s ==============================
PS F:\ProcuraMed>
```

Figure 1 : Purchase Request Submission Test successfully (Pytest Execution Results)

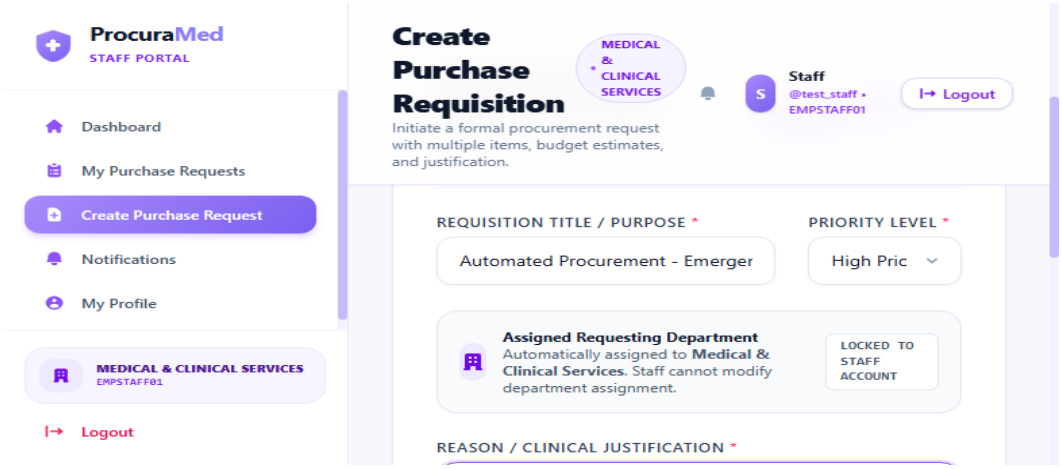


Figure 2 : Step 1 - General Requisition Details Entered

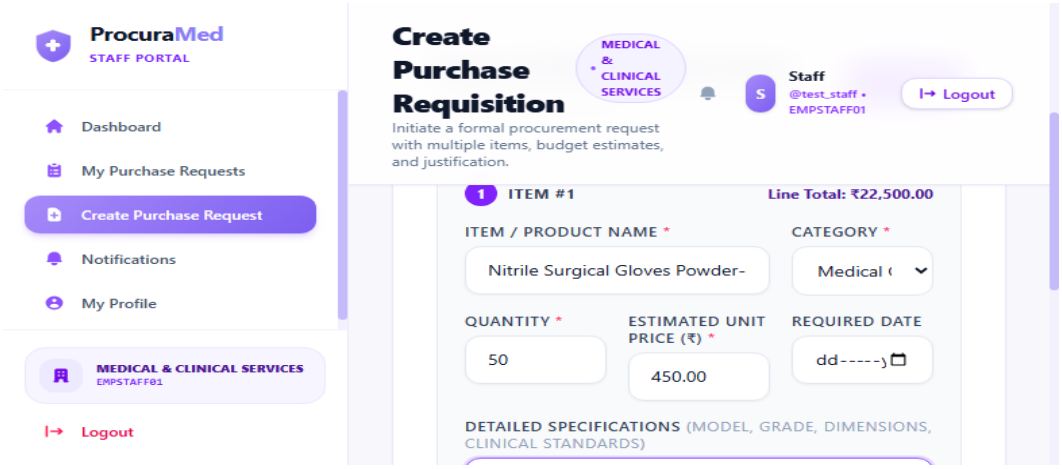


Figure 3 : Step 2 - Item Specifications and Budget Details Entered

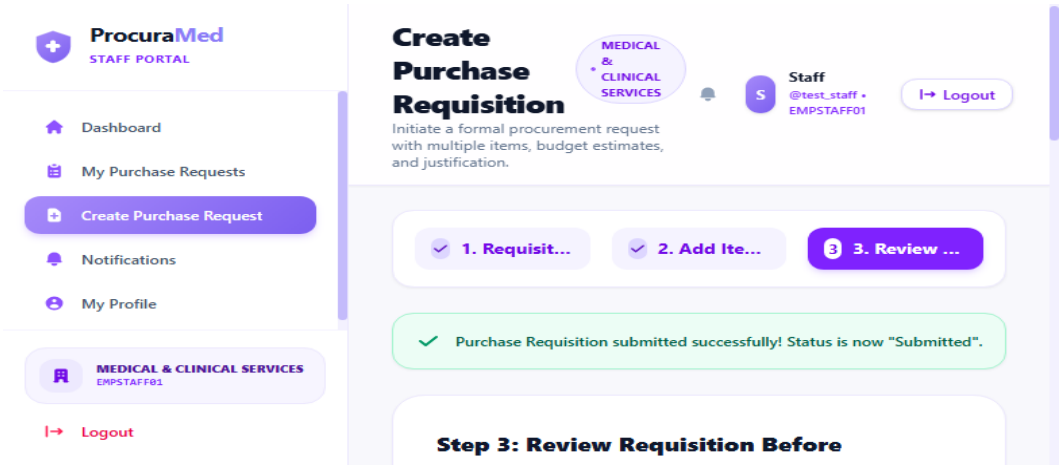


Figure 4 : Step 3 - Requisition Submission Confirmation Notification

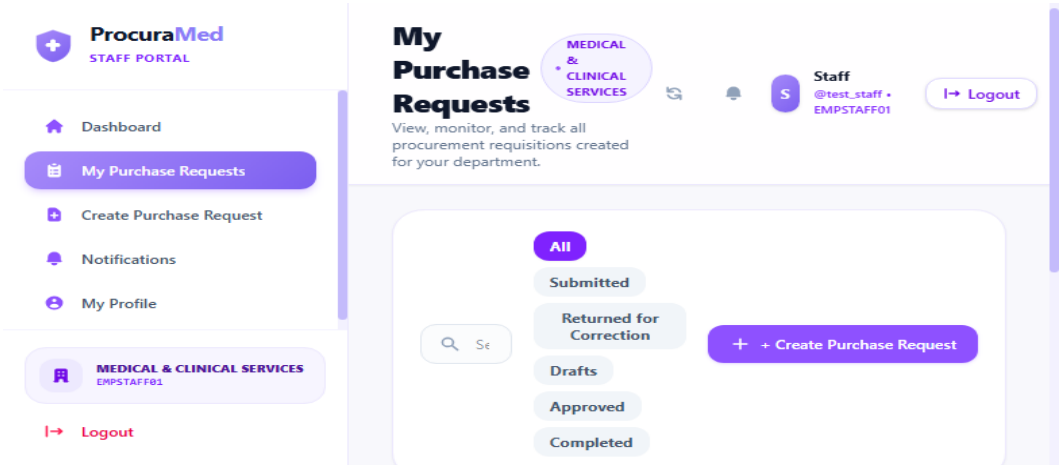


Figure 5 : Verified Requisition Entry with Status 'Submitted' in My Requisitions List

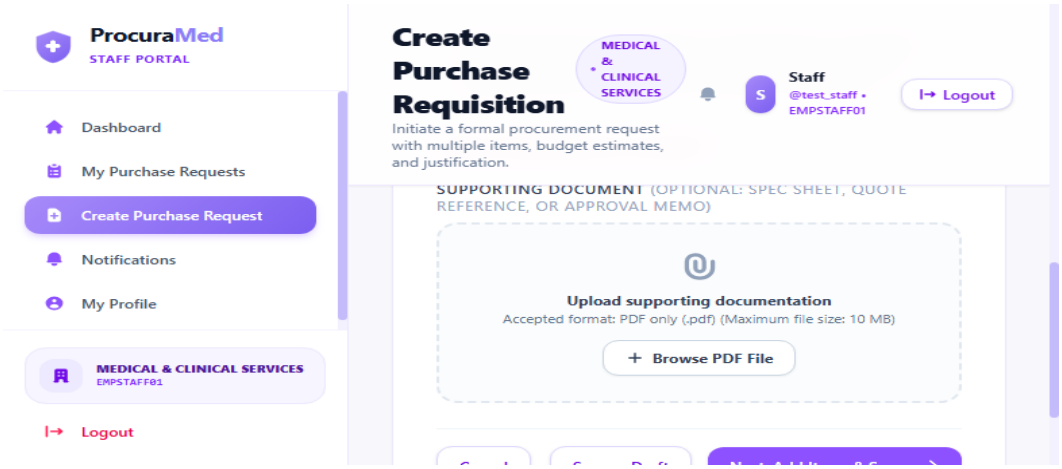


Figure 6 : Required Form Field Validation Alerts

Test Report
Test Case 2

Project Name: ProcuraMed					
Purchase Request Submission Test Case					
Test Case ID: Test_2			**Test Designed By:** QA Automation Engineer		
Test Priority(Low/Medium/High): High			**Test Designed Date:** 22/10/2026		
Module Name: Purchase Requisition			**Test Executed By :** QA Automation Engineer		
Test Title : verify creation and submission of purchase request with valid item specifications.			**Test Execution Date:** 22/10/2026		
Description: User has valid requester credentials and enters complete item specifications for procurement.					
Pre-Condition : Authorized Department Staff user is logged into ProcuraMed portal					
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status(Pass/Fail)**
1	Navigate to Create Purchase Requisition page	N/A	Display Create Purchase Requisition form	Create Purchase Requisition form displayed	**pass**
2	Enter general requisition details	Title: Automated Procurement... Priority: HIGH Justification: Emergency stock...	Requisition details accepted; proceed to item specifications	Requisition details accepted; proceed to item specifications	**pass**
3	Enter item specifications	Item: Nitrile Surgical Gloves Category: Medical Consumables Qty: 50, Price: 450.00	Item details added to requisition list	Item details added to requisition list	**pass**
4	Review requisition and click submit button	N/A	Purchase requisition submitted successfully; confirmation message displayed and listed under My Requisitions	Purchase Requisition submitted successfully! Status is now "Submitted"; listed under My Requisitions	**pass**
Post-Condition: Requisition record is saved in database with status 'SUBMITTED' and assigned unique requisition tracking number.					

Test Report – Negative Test Cases
Test Case 2.1: Required Field Validation

Project Name: ProcuraMed					
Purchase Request Field Validation Test Cases					
Test Case ID: Test_2_Neg			**Test Designed By:** QA Automation Engineer		
Test Priority(Low/Medium/High): Medium			**Test Designed Date:** 22/10/2026		
Module Name: Purchase Requisition			**Test Executed By :** QA Automation Engineer		
Test Title : verify validation error messages when mandatory fields are omitted.			**Test Execution Date:** 22/10/2026		
Description: Verify form validation prevents submission when required title and clinical justification fields are omitted.					
Pre-Condition : User is on the ProcuraMed Create Purchase Requisition page					
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status(Pass/Fail)**
1	Navigate to Create Purchase Requisition page	N/A	Display Create Purchase Requisition form	Form loaded successfully	**pass**
2	Attempt proceeding with blank Title and Justification fields	Title: [Empty] Justification: [Empty]	Validation error messages displayed: 'Requisition title / purpose is required.' and 'Reason and justification are required.'	Validation errors displayed correctly; navigation to Step 2 blocked	**pass**
Post-Condition: Form submission is blocked; no invalid requisition is committed to database.					